

華中科技大學

课程实验报告

课程名称： 深度学习

实验名称： 实验四 DQN

院 系： 计算机科学与技术学院

专业班级： 本硕博 2301 班

学 号： U202315763

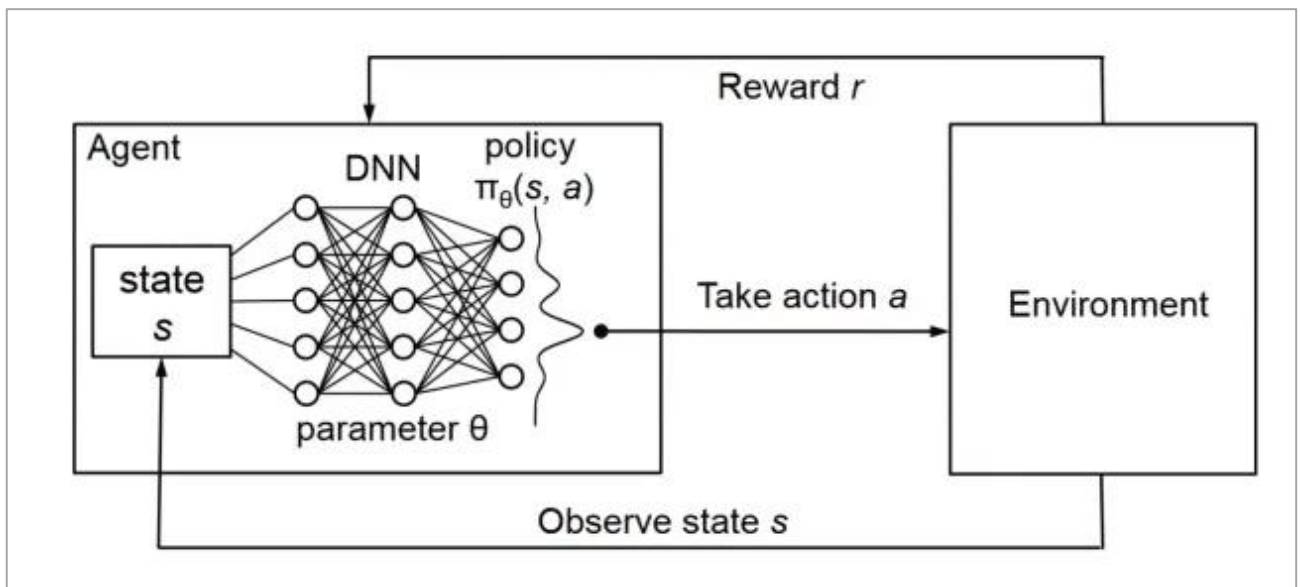
姓 名： 王家乐

2026 年 1 月 13 日

1. 实验目的与环境

本实验旨在基于深度强化学习方法实现并验证深度 Q 网络（Deep Q-Network, DQN）在 CartPole-v1 经典控制任务上的有效性。通过构建以神经网络近似动作价值函数 $Q(s,a)$ 的智能体，并结合经验回放与目标网络等稳定训练机制，使智能体能够学习到稳定的控制策略，从而在小车倒立摆环境中尽可能长时间保持杆子直立。

实验目标为达到环境的通关标准：在训练过程中，最近连续 100 个回合的平均回报（episode reward）不低于 195 分。为满足该目标，实验将完整实现 DQN 训练流程，包括与环境交互采样、经验存储与随机回放、网络参数迭代更新，以及探索策略随训练逐步衰减等关键环节，并通过训练曲线可视化对收敛过程与稳定性进行评估。



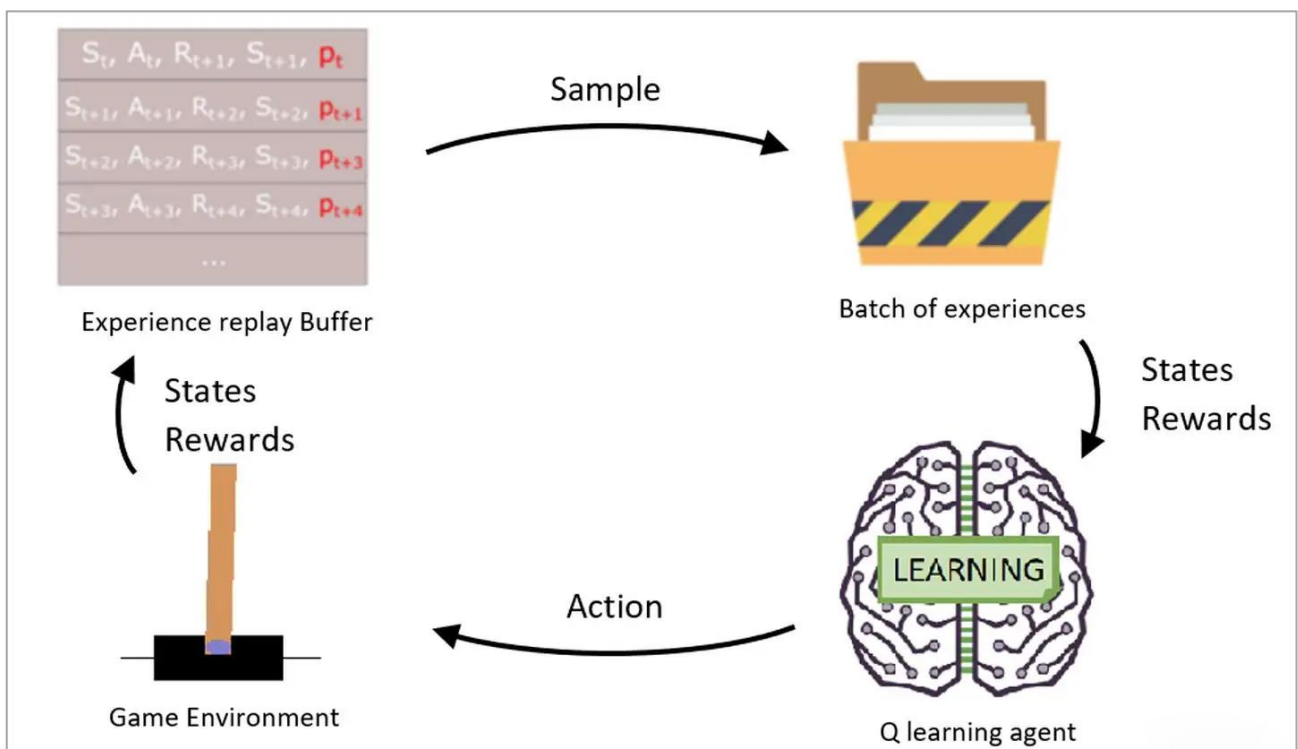
实验环境采用 Gymnasium 提供的 CartPole-v1 环境。该环境的状态空间为 4 维连续向量，通常包含小车位置、小车速度、杆角度与杆角速度等信息；动作空间为 2 个离散动作，分别对应对小车施加向左或向右的力。每个回合从初始

状态开始，智能体根据当前状态选择动作并获得即时奖励，直到杆角度或小车位置超出阈值、或回合达到最大步数限制为止。实验以回合累计奖励作为性能指标，并以 100 回合滑动平均回报作为是否达标的判据。

实现层面，使用 Python 与 PyTorch 搭建 Q 网络并完成训练。训练与评估过程中，将记录每回合回报、损失变化以及探索率等指标，并以曲线形式呈现，以支持对学习动态、收敛速度与最终性能的定量分析。

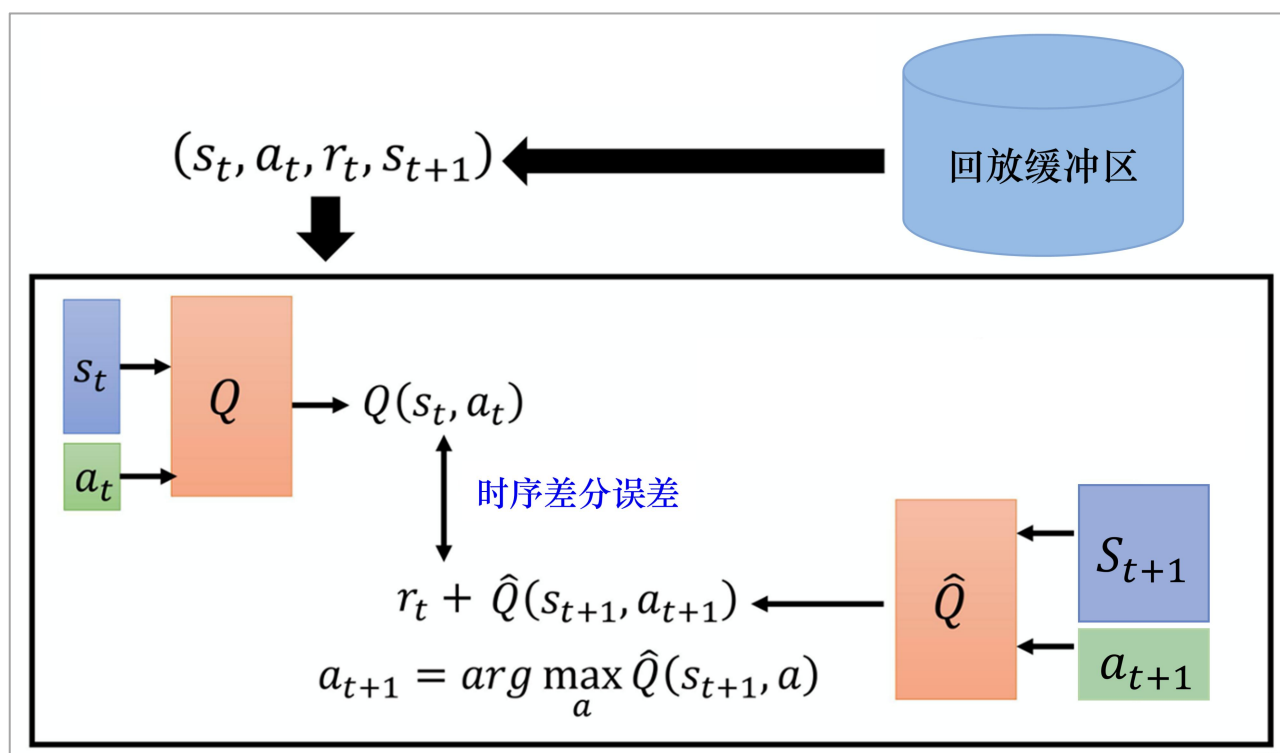
2. DQN 算法简介

DQN（Deep Q-Network）的核心思想是用深度神经网络近似动作价值函数 $Q(s,a)$ ，从而在高维或连续状态空间中实现基于价值迭代的强化学习。对于每个状态 (s) ，网络输出对应各个离散动作的 Q 值估计，智能体依据这些估计选择动作，并通过与环境交互获得的奖励信号不断修正 Q 值预测，使得在长期回报意义下更优的动作得到更高的价值评估。



在更新机制上，DQN 沿用 Q-learning 的时序差分（Temporal Difference, TD）思想：利用一步 bootstrapping 构造目标值，将当前 Q 估计向“即时奖励 + 折扣后的下一状态最优价值”逼近。具体而言，网络以 $(s, a, r, s', done)$ 的交互样本为基础，通过最小化当前 Q 值与 TD 目标之间的差异（常用均方误差或 Huber 损失）来进行梯度下降更新。

为解决样本相关性与训练发散问题，DQN 引入了两项关键机制。第一是经验回放（Experience Replay）：将交互过程中产生的转移样本存入回放缓冲区，训练时从中随机采样小批量数据进行更新。随机采样能够打破时间序列相关性，提高训练的稳定性，并提升样本利用效率。第二是目标网络（Target Network）：维护一份相对滞后的网络参数用于计算 TD 目标，将“目标值的生成”与“当前网络的更新”解耦，显著减轻训练过程中目标不断漂移带来的不稳定。



在动作选择策略方面，DQN 通常采用 ϵ -greedy 机制实现探索与利用的折中：以概率 ϵ 随机选择动作以探索未知状态-动作对，以概率 $(1-\epsilon)$ 选择当前

Q 值最大的动作以利用已学策略。训练初期设置较大的 ϵ 以促进充分探索，随后逐步衰减 ϵ 以提高策略的确定性与性能稳定性。通过“函数逼近 + TD 学习 + 回放缓冲 + 目标网络 + ϵ -greedy 探索”的组合，DQN 能够在经典控制等离散动作任务中实现较为稳定且有效的端到端学习。

```
p = random()
if p <  $\epsilon$  :
    pull random action
else:
    pull current-best action
```

3. 实现细节

3.1 Q 网络结构

本实验的 Q 网络用于近似动作价值函数 $Q(s,a)$ ，采用多层感知机（MLP）的全连接结构实现从环境状态到各动作 Q 值的映射。网络输入为 CartPole-v1 的 4 维状态向量（小车位置、小车速度、杆角度、杆角速度），输出为 2 维向量，对应两个离散动作（向左/向右施力）的价值估计。智能体在决策时通过比较输出的两个 Q 值大小选择当前更优动作。

网络主体由两层隐藏层构成，隐藏单元规模设置为 128 和 128。隐藏层采用 ReLU 非线性激活函数，以增强网络的表达能力并提升训练阶段的梯度传播效果。输出层为线性层，不使用激活函数，直接产生每个动作的 Q 值估计，便于与 TD 目标进行回归拟合。

在实现上，主网络（local network）与目标网络（target network）采用完全相同的网络结构与参数维度，保证二者在同一函数族内进行估计与对齐更新。主网络负责学习并参与动作选择，目标网络用于计算相对稳定的目标值（并在本实验

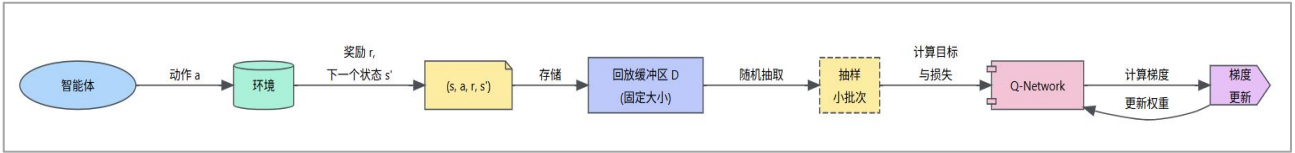
中配合 Double DQN 进行动作评估)，从结构层面支持训练稳定性与收敛表现。

```
class QNetwork(nn.Module):
    def __init__(self, state_dim: int, action_dim: int, hidden_sizes=(128, 128)):
        super().__init__()
        layers = []
        in_dim = state_dim
        for h in hidden_sizes:
            layers += [nn.Linear(in_dim, h), nn.ReLU()]
            in_dim = h
        layers += [nn.Linear(in_dim, action_dim)]
        self.net = nn.Sequential(*layers)
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.kaiming_uniform_(m.weight, a=math.sqrt(5))
                if m.bias is not None:
                    fan_in, _ = nn.init._calculate_fan_in_and_fan_out(m.weight)
                    bound = 1 / math.sqrt(fan_in)
                    nn.init.uniform_(m.bias, -bound, bound)
    def forward(self, x):
        return self.net(x)
```

3.2 经验回放 ReplayBuffer

经验回放（Replay Buffer）的设计目的是缓解强化学习中“在线采样”带来的强序列相关性问题，并提升样本利用效率。在与环境交互过程中，智能体会连续产生转移样本 $(s, a, r, s', done)$ 。如果直接按时间顺序用这些样本更新网络，梯度更新会受到相邻样本高度相关的影响，容易导致训练不稳定甚至发散。Replay Buffer 通过将交互经验集中存储并在训练时随机抽取，能够有效打破样本的时间相关性，使更新过程更接近独立同分布假设，从而提高 DQN 的稳定性。

本实验中，Replay Buffer 采用固定容量的队列结构实现，容量达到上限后“先进先出”的方式覆盖旧样本，确保缓冲区中始终保存近期与多样化的交互经验。每次环境交互结束后，将当前状态、执行动作、获得奖励、下一状态以及终止标志组成五元组写入缓冲区，为后续训练提供数据来源。



在参数更新时，缓冲区会随机采样一个小批量 (mini-batch) 样本用于计算 TD 目标并更新主网络参数。随机采样不仅提高了样本复用率，也使得每次更新所依据的数据更加多样化。实现上，采样得到的批数据会被整理为张量并移动到相同计算设备上，与 Q 网络前向计算、损失函数与反向传播流程无缝衔接，形成完整的“交互—存储—随机回放—更新”的训练闭环。

```

Experience = namedtuple("Experience", ["state", "action", "reward", "next_state", "done"])

class ReplayBuffer:
    def __init__(self, capacity: int, batch_size: int, seed: int = 0):
        self.buffer = deque(maxlen=capacity)
        self.batch_size = batch_size
        random.seed(seed)

    def push(self, state, action, reward, next_state, done):
        self.buffer.append(Experience(state, action, reward, next_state, done))

    def sample(self):
        batch = random.sample(self.buffer, k=self.batch_size)
        states = torch.from_numpy(np.vstack([e.state for e in batch])).float().to(device)
        actions = torch.from_numpy(np.vstack([e.action for e in batch])).long().to(device)
        rewards = torch.from_numpy(np.vstack([e.reward for e in batch])).float().to(device)
        next_states = .....
        dones = .....
        return states, actions, rewards, next_states, dones

    def __len__(self):
        return len(self.buffer)
    
```

3.3 DQN 智能体

本实验的 DQN 智能体由“策略执行 (action selection)”与“价值学习 (value learning)”两部分构成，整体围绕主网络 (local network) 与目标网络 (target network) 的双网络架构展开。主网络用于近似当前的 $Q(s,a)$ 并参与动作选择，目标网络用于计算相对稳定的 TD 目标值，从而降低训练过程中目标漂移导致的震荡与

发散风险。两者结构完全一致，但参数更新策略不同：主网络通过梯度下降频繁更新，目标网络通过软更新缓慢跟随主网络，以实现稳定学习。

在交互与决策层面，智能体采用 ϵ -greedy 策略平衡探索与利用：以概率 ϵ 随机选择动作以探索状态空间，以概率 $(1-\epsilon)$ 选择主网络输出 Q 值最大的动作以利用当前策略。训练初期 ϵ 较大保证充分探索，随着训练推进逐步衰减到设定下限，使智能体逐渐转向稳定利用已学到的高价值动作，从而提升回报并减少后期波动。该策略与 **CartPole-v1** 的离散动作特性匹配良好，能够在学习初期快速覆盖有效动作序列。

在学习更新层面，智能体通过经验回放缓冲区获取随机小批量样本进行训练。对每个 batch，主网络输出当前状态下所选动作的 Q 值估计；目标值部分则在 DQN 框架下进一步采用 Double DQN 计算：先由主网络在下一状态 s' 上选择 a' ，再由目标网络评估，并与即时奖励及终止标志共同构成 TD 目标。这种“选择与评估分离”的方式可以减轻 Q 值过估计问题，带来更稳定的收敛表现。

优化方面，智能体使用 Adam 优化器对主网络参数进行更新，折扣因子 (γ) 控制对未来回报的权重。损失函数采用 Huber Loss (Smooth L1)，相较均方误差对异常 TD 误差更鲁棒，有助于避免训练早期因误差过大导致梯度不稳定。同时引入梯度裁剪限制梯度范数，进一步降低数值不稳定风险。每次学习后对目标网络执行软更新（以较小 τ 将主网络参数缓慢融合到目标网络），从而在保证稳定性的同时持续提升策略性能，形成完整且可复现实验结果的 DQN 智能体设计。

```
class DQNAgent:
    def __init__(
        self, state_dim: int, action_dim: int,
        lr: float = 1e-3, gamma: float = 0.99,
        buffer_size: int = 100_000,
```



```

batch_size: int = 64, tau: float = 1e-3,
update_every: int = 1, seed: int = 0,
):
    self.state_dim = state_dim
    self.action_dim = action_dim
    self.gamma = gamma
    self.batch_size = batch_size
    self.tau = tau
    self.update_every = update_every
    self.t_step = 0
    self.q_local = QNetwork(state_dim, action_dim).to(device)
    self.q_target = QNetwork(state_dim, action_dim).to(device)
    self.q_target.load_state_dict(self.q_local.state_dict())
    self.q_target.eval()
    self.optimizer = optim.Adam(self.q_local.parameters(), lr=lr)
    self.memory = ReplayBuffer(buffer_size, batch_size, seed=seed)
    self.loss_fn = nn.SmoothL1Loss() # Huber loss

def act(self, state: np.ndarray, eps: float = 0.0) -> int:
    """ε-greedy 选动作"""
    if random.random() < eps:
        return random.randrange(self.action_dim)
    state_t = torch.from_numpy(state).float().unsqueeze(0).to(device)
    self.q_local.eval()
    with torch.no_grad():
        q_values = self.q_local(state_t)
    self.q_local.train()
    return int(torch.argmax(q_values, dim=1).item())

def step(self, state, action, reward, next_state, done):
    self.memory.push(state, action, reward, next_state, done)
    self.t_step = (self.t_step + 1) % self.update_every
    if self.t_step == 0 and len(self.memory) >= self.batch_size:
        return self.learn(self.memory.sample())
    return None

def learn(self, experiences):
    states, actions, rewards, next_states, dones = experiences
    # 当前 Q(s,a)
    q_expected = self.q_local(states).gather(1, actions)
    # Double DQN:
    # 1) 用主网络在 next_states 上选 a' = argmax_a Q_local(next_state, a)
    # 2) 用目标网络估值 Q_target(next_state, a')
    with torch.no_grad():
        next_actions = torch.argmax(self.q_local(next_states), dim=1, keepdim=True)

```

```

        q_next = self.q_target(next_states).gather(1, next_actions)
        q_target = rewards + (1 - dones) * self.gamma * q_next
        loss = self.loss_fn(q_expected, q_target)
        self.optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(self.q_local.parameters(), max_norm=10.0)
        self.optimizer.step()
        # 软更新目标网络
        self.soft_update(self.q_local, self.q_target, self.tau)
        return float(loss.item())

    @staticmethod
    def soft_update(local_model, target_model, tau: float):
        for target, local in zip(target_model.parameters(), local_model.parameters()):
            target.data.copy_(tau * local.data + (1.0 - tau) * target.data)

```

3.4 超参数

本实验的超参数设置以“稳定收敛并满足 CartPole-v1 通关标准”为原则：学习率取 0.001 并配合 Adam 优化器，在保证收敛速度的同时避免更新过激；折扣因子 $\gamma=0.99$ 强调长期回报，使策略更关注持续平衡的累积收益；经验回放缓冲区容量设为 100000 以覆盖足够多样的状态转移并提升样本复用率，批量大小设为 64 在梯度估计稳定性与计算效率间取得折中；目标网络采用软更新 $\tau=1 \times 10^{-3}$ 以缓慢跟随主网络、降低目标漂移；探索策略采用 ϵ -greedy (ϵ 从 1.0 衰减至 0.05，衰减系数 0.995) 以保证前期充分探索、后期稳定利用；同时使用 Huber 损失与梯度裁剪（最大范数 10）增强对异常 TD 误差的鲁棒性，综合提升训练稳定性与最终性能。

4. 结果与分析

训练流程以“回合级训练 + 步级交互更新”为主线展开。本实验设置最多训练回合数为 $n_episodes=2000$ ，每个回合的最大交互步数为 $max_t=500$ 。在每个回

合开始时，智能体从环境获取初始状态；随后在回合内部按时间步循环：依据当前状态通过 ϵ -greedy 策略选择动作，与环境交互得到下一状态、即时奖励与终止标志，并将转移样本 $(s,a,r,s',done)$ 存入经验回放缓冲区，直至回合终止或达到最大步数上限。训练采用随回合衰减的 ϵ -greedy 参数：`eps_start=1.0` 表示训练初期以较高概率随机探索，`eps_end=0.05` 作为探索下限避免完全贪心导致的局部最优风险，`eps_decay=0.995` 控制 ϵ 的衰减速度。

训练过程中按固定间隔输出训练日志以监控收敛情况，设置 `print_every=20`，即每 20 个回合打印一次当前回合回报、最近 100 回合平均回报以及 ϵ 等信息，便于观察性能提升趋势与探索率变化。最终以“最近 100 回合滑动平均回报是否达到 195”为主要成功判据，并结合 reward、loss 与 ϵ 曲线可视化对训练过程的稳定性与收敛行为进行分析。训练过程 log 如下：

```
Episode 20 | Avg(100) = 21.30 | eps = 0.905
Episode 40 | Avg(100) = 20.12 | eps = 0.818
Episode 60 | Avg(100) = 20.92 | eps = 0.740
Episode 80 | Avg(100) = 20.51 | eps = 0.670
Episode 100 | Avg(100) = 22.55 | eps = 0.606
Episode 120 | Avg(100) = 25.08 | eps = 0.548
Episode 140 | Avg(100) = 27.92 | eps = 0.496
Episode 160 | Avg(100) = 36.38 | eps = 0.448
Episode 180 | Avg(100) = 42.66 | eps = 0.406
Episode 200 | Avg(100) = 49.50 | eps = 0.367
Episode 220 | Avg(100) = 58.54 | eps = 0.332
Episode 240 | Avg(100) = 70.34 | eps = 0.300
Episode 260 | Avg(100) = 76.93 | eps = 0.272
Episode 280 | Avg(100) = 82.65 | eps = 0.246
Episode 300 | Avg(100) = 88.63 | eps = 0.222
Episode 320 | Avg(100) = 92.06 | eps = 0.201
Episode 340 | Avg(100) = 101.91 | eps = 0.182
Episode 360 | Avg(100) = 112.24 | eps = 0.165
Episode 380 | Avg(100) = 116.91 | eps = 0.149
Episode 400 | Avg(100) = 120.84 | eps = 0.135
Episode 420 | Avg(100) = 131.37 | eps = 0.122
Episode 440 | Avg(100) = 128.36 | eps = 0.110
Episode 460 | Avg(100) = 119.18 | eps = 0.100
```

```

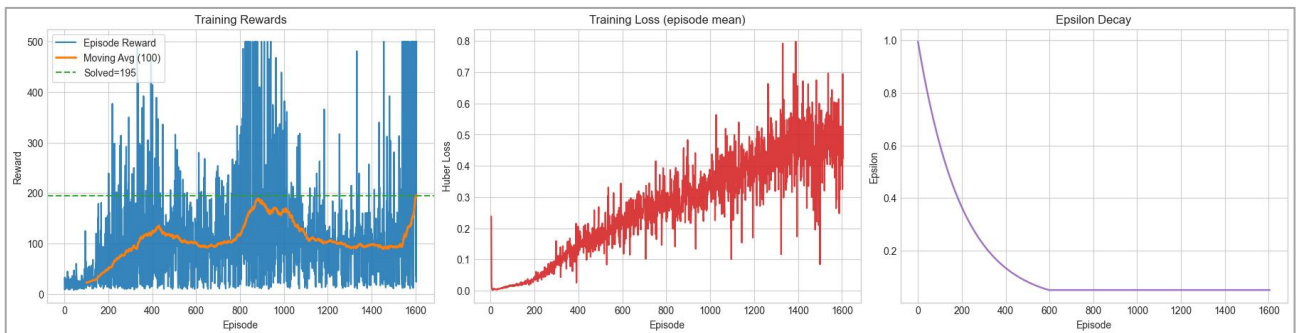
Episode 480 | Avg(100) = 118.00 | eps = 0.090
Episode 500 | Avg(100) = 113.12 | eps = 0.082
Episode 520 | Avg(100) = 111.24 | eps = 0.074
Episode 540 | Avg(100) = 103.10 | eps = 0.067
Episode 560 | Avg(100) = 103.16 | eps = 0.060
Episode 580 | Avg(100) = 103.12 | eps = 0.055
Episode 600 | Avg(100) = 102.24 | eps = 0.050
Episode 620 | Avg(100) = 94.26 | eps = 0.050
Episode 640 | Avg(100) = 95.46 | eps = 0.050
Episode 660 | Avg(100) = 93.77 | eps = 0.050
Episode 680 | Avg(100) = 94.67 | eps = 0.050
Episode 700 | Avg(100) = 100.00 | eps = 0.050
Episode 720 | Avg(100) = 100.92 | eps = 0.050
Episode 740 | Avg(100) = 100.82 | eps = 0.050
Episode 760 | Avg(100) = 102.30 | eps = 0.050
Episode 780 | Avg(100) = 106.95 | eps = 0.050
Episode 800 | Avg(100) = 115.11 | eps = 0.050
Episode 820 | Avg(100) = 133.70 | eps = 0.050
Episode 840 | Avg(100) = 160.33 | eps = 0.050
Episode 860 | Avg(100) = 171.17 | eps = 0.050
Episode 880 | Avg(100) = 185.92 | eps = 0.050
Episode 900 | Avg(100) = 185.72 | eps = 0.050
Episode 920 | Avg(100) = 173.91 | eps = 0.050
Episode 940 | Avg(100) = 160.38 | eps = 0.050
Episode 960 | Avg(100) = 168.87 | eps = 0.050
Episode 980 | Avg(100) = 160.53 | eps = 0.050
Episode 1000 | Avg(100) = 162.86 | eps = 0.050
Episode 1020 | Avg(100) = 162.16 | eps = 0.050
Episode 1040 | Avg(100) = 150.82 | eps = 0.050
Episode 1060 | Avg(100) = 132.60 | eps = 0.050
Episode 1080 | Avg(100) = 129.46 | eps = 0.050
Episode 1100 | Avg(100) = 116.35 | eps = 0.050
Episode 1120 | Avg(100) = 108.17 | eps = 0.050
Episode 1140 | Avg(100) = 108.51 | eps = 0.050
Episode 1160 | Avg(100) = 109.69 | eps = 0.050
Episode 1180 | Avg(100) = 101.75 | eps = 0.050
Episode 1200 | Avg(100) = 103.36 | eps = 0.050
Episode 1220 | Avg(100) = 103.85 | eps = 0.050
Episode 1240 | Avg(100) = 102.55 | eps = 0.050
Episode 1260 | Avg(100) = 100.89 | eps = 0.050
Episode 1280 | Avg(100) = 98.01 | eps = 0.050
Episode 1300 | Avg(100) = 92.92 | eps = 0.050
Episode 1320 | Avg(100) = 93.87 | eps = 0.050
Episode 1340 | Avg(100) = 95.62 | eps = 0.050
Episode 1360 | Avg(100) = 96.27 | eps = 0.050

```

Episode 1380 | Avg(100) = 95.40 | eps = 0.050
 Episode 1400 | Avg(100) = 98.37 | eps = 0.050
 Episode 1420 | Avg(100) = 95.90 | eps = 0.050
 Episode 1440 | Avg(100) = 95.78 | eps = 0.050
 Episode 1460 | Avg(100) = 92.84 | eps = 0.050
 Episode 1480 | Avg(100) = 91.07 | eps = 0.050
 Episode 1500 | Avg(100) = 93.04 | eps = 0.050
 Episode 1520 | Avg(100) = 93.90 | eps = 0.050
 Episode 1540 | Avg(100) = 93.92 | eps = 0.050
 Episode 1560 | Avg(100) = 118.15 | eps = 0.050
 Episode 1580 | Avg(100) = 136.38 | eps = 0.050
 Episode 1600 | Avg(100) = 184.90 | eps = 0.050

Solved! Episode 1606 | Avg(100) = 196.16

训练结束后，将奖励曲线、100 回合滑动平均、损失曲线、 ϵ 曲线可视化：



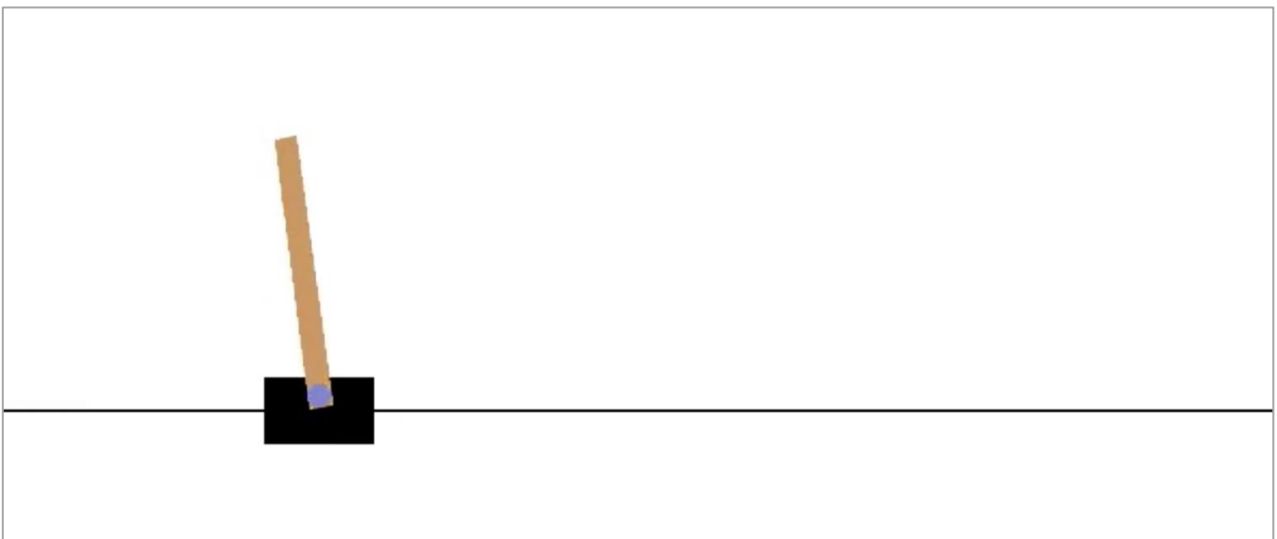
从训练日志与曲线整体趋势看，智能体在训练早期（约 0 - 400 回合）处于典型的“高探索、低回报”阶段，最近 100 回合平均回报从约 20 分逐步提升到 120 分左右；此阶段 ϵ 仍处于较高水平（例如第 200 回合 $\epsilon \approx 0.367$ ），策略尚未稳定，但已能学习到基本的保持平衡行为。

随着训练推进与探索率衰减，性能出现显著跃迁： ϵ 按设定衰减并在约第 600 回合达到下限 0.05 后保持不变（日志中第 600 回合 $\text{eps}=0.050$ ），此后滑动平均回报在 800 - 900 回合附近快速上升，最高接近通关阈值（例如第 880 回合 $\text{Avg}(100)=185.92$ ）。奖励曲线中也可见单回合回报频繁出现高值，甚至多次触及上限 500，说明智能体已能在部分回合实现长时间稳定控制，但整体仍存在较大波动。

中后期训练呈现“性能上升—回落—再恢复”的非单调特征：在 900 回合之后， $\text{Avg}(100)$ 从约 185 附近回落并在较长区间内徘徊于约 90 - 160 之间（例如第 1100 回合 $\text{Avg}(100)=116.35$ 、第 1300 回合 $\text{Avg}(100)=92.92$ ），表明在固定较低 ϵ 的情况下，策略仍可能因价值估计误差累积、分布漂移等因素出现阶段性退化；不过后续训练再次获得有效提升，并在第 1606 回合达到通关标准——最近 100 回合平均回报 $196.16 \geq 195$ ，满足实验要求。

损失曲线方面，Huber loss 的回合均值总体呈上升并伴随较强抖动，这与 DQN 训练中目标值分布、Q 值尺度变化以及自举目标引入的非平稳性一致。需要强调的是，TD 损失与策略实际回报并非严格单调对应：在回报提升或波动阶段，损失仍可能维持较高水平甚至上升，因此本实验中应以“回报曲线与 100 回合滑动平均是否越过 195”作为主要达标依据，同时结合损失曲线判断训练是否出现数值不稳定（例如异常爆炸）即可。

为了评估效果，将训练完成的智能体在 CartPole-v1 环境中的表现录制成视频并保存到本地指定的 videos 文件夹，使用环境自带的视频录制包装器在若干个评估回合中对智能体进行“纯贪心”控制,视频如下（ctrl+鼠标左键打开视频）：



5. 总结

本实验围绕 CartPole-v1 任务完整实现了 DQN 训练闭环：以 MLP 近似动作价值函数 ($Q(s,a)$)，结合经验回放打破样本相关性、目标网络（软更新）稳定 TD 目标，并采用 ϵ -greedy 实现探索—利用折中；在更新层面进一步引入 Double DQN 的“动作选择—动作评估”分离、Huber Loss 与梯度裁剪以提升训练鲁棒性。训练过程中，滑动平均回报整体呈“低回报探索期—性能跃迁—波动回落—再次恢复”的典型强化学习动态特征，最终在第 1606 回合达到最近 100 回合平均回报 196.16，满足通关判据 (≥ 195)，验证了所实现 DQN 方案在该经典控制问题上的有效性。

从结果分析看， ϵ 衰减至下限后仍出现较长区间的性能回落，反映出自举目标带来的非平稳性、价值估计误差累积以及数据分布漂移对训练稳定性的影响；因此，实验评价应以回报及其滑动平均为核心指标，损失曲线更多用于监测数值稳定性而非单调性能判定。总体而言，本实验不仅达成了预设性能目标，也呈现了 DQN 在中后期可能出现的震荡现象，为后续进一步提升稳定性（如更精细的学习率/更新频率设置、改进探索策略或回放采样策略等）提供了明确的观察依据与优化方向。